

```
        else:
            return False

    def getArea(self):
        theArea = .5 * self.width * self.height
        return theArea

    def draw(self):
        pygame.draw.polygon(self.window, self.color, (
            (self.x, self.y + self.height),
            (self.x, self.y),
            (self.x + self.width, self.y)))
```

Listing 10-9: The `Triangle` class that inherits from the `Shape` class

The main code we used for testing in Chapter 9 doesn't have to change at all. As a client of these new classes, it instantiates `Square`, `Circle`, and `Triangle` objects without having to worry about the implementation of those classes. It doesn't need to know that each is subclassed from a common `Shape` class.

Abstract Classes and Methods

Unfortunately, our `Shape` base class has a potential bug. At the moment, a client could instantiate a generic `Shape` object, which is too generic to have its own `getArea()` method. Further, all classes that inherit from the `Shape` class (like `Square`, `Circle`, and `Triangle`) *must* implement `clickedInside()`, `getArea()`, and `draw()`. To solve both of these problems, I'll introduce the concepts of an *abstract class* and an *abstract method*.

abstract class

A class that is *not* intended to be instantiated directly, but only to be used as a base class by one or more subclasses. (In some other languages, an abstract class is referred to as a *virtual class*.)
